Replica Management

Key Issues of replica management

- Where, when and by whom replicas should be placed.
- Mechanisms to keep them consistent.
- Two main sub-problems:
  - Replica-server Placement
    - Finding best location and find where a server can be placed.
  - Content Placement
    - Finding out which server is best for storing a particular content.

Replica Server Placement

- Based on Distance between clients and locations as starting point.
  - Best K out of N locations (K<N) are selected
  - One of the method used for replica server placements is based on the distance between clients and locations as starting point. Distance can be measured in terms of latency or bandwidth.
  - The solution selects one server at a time such that the average distance between that server and its clients is minimal given that already k servers have been placed (meaning that there are N - k locations left).
- By applying Clustering
  - Group nodes accessing the same content and with low inter-nodal latencies into groups or clusters and place a replica on the k largest clusters.

Content Replication and Placement

- Permanent Replicas
  - Geographically distributed - Mirroring
  - Same location – Round Robin
- Server Initiated Replicas
  - To enhance performance, a data store dynamically place a replica in a geographical location to meet the unexpected burst of request coming from that location (also known as push caches)
- Client Initiated Replicas
  - A type of replica that is created at the initiative of clients (also known as client caches)

Permanent Replicas

- Permanent replicas can be considered as the initial set of replicas that constitute a distributed data store.
- In many cases, the number of permanent replicas is small.
- Consider, for example, a Web site: Distribution of a Web site generally comes in one of two forms.
- The first kind of distribution is one in which the files that constitute a site are replicated across a limited number of servers at a **single location.** Whenever a request comes in, it is forwarded to one of the servers, for instance, using a round-robin strategy.
- The second form is called **mirroring**. In this case, a Web site is copied to a limited number of servers, called mirror sites, which are geographically spread across the Internet. In most cases, clients simply choose one of the various mirror sites from a list offered to them. Mirrored websites have in common with cluster-based Web sites that there are only a few number of replicas, which are more or less statically configured.

Server Initiated Replicas

- Initiative of owner of data store
- Enhance performance
- Example: Web hosting services
  - can dynamically replicate files to servers where those files are needed most to enhance performance or to reduce the load on servers
- Where and when replicas should be created and deleted?
  - Replicate (or migrate) files to servers that are close to clients that issue many requests
  - Each server keeps track of access count per file, and where those access requests come from
  - Each server can determine which of the servers in the Web hosting service is closest to the client C

Server Initiated Replicas

- Condition for creating a replica at P
  - When $cnt_Q(P, F)$ exceeds a certain threshold, say rep(P, F).
- Condition for deleting the replica at P
  - When requests for a file F at server P drops below a threshold, say del(P, F).
  - Deletion is executed only if the copy is not the last one
  - Deletion threshold is chosen much lower than the replication threshold.
- If $cnt_Q(P, F)$ exceeds more than half of the total requests for F at Q, server P is requested to take over the copy of F (migration)

Client Initiated Replicas

- Client-initiated replicas are more commonly known as client caches.
- Managing is entirely by client
- Client caches are used only to improve access times to data. Normally, when a client wants access to some data, it connects to the nearest copy of the data store from where it fetches the data it wants to read, or to where it stores the data it had just modified.
- Data are generally kept in a cache for a limited amount of time, for example, to prevent extremely stale data from being used, or simply to make room for other data.
- To improve the number of cache hits, caches can be shared between clients. The underlying assumption is that a data request from client C1 may also be useful for a request from another nearby client C2
- Placement
  - Same machine
  - LAN
  - WAN

Content Distribution

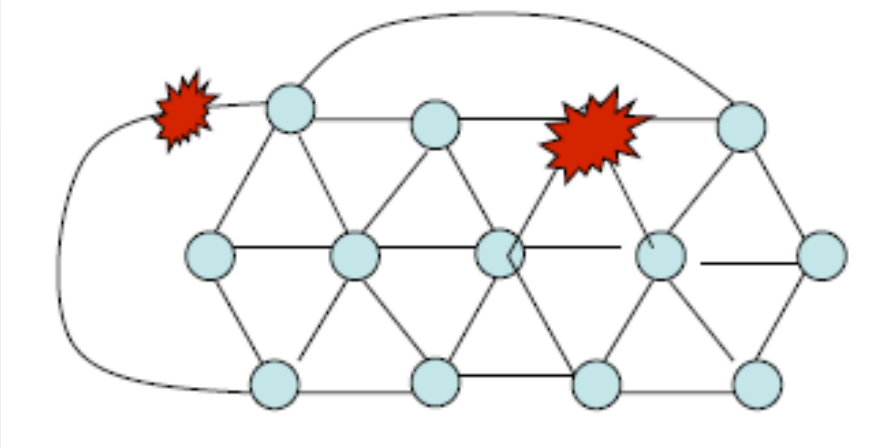
- Propagation of Updated content
  - Clients initiate
  - They are subsequently forwarded to one of the copies
  - From there, the update should be propagated to other copies, while ensuring the consistency at the same time
- What to propagate?
  1. Only a notification of an update (e.g. invalidation used for caches). Useful when read-to-update ratio is low
  2. Transfer the modified data from one copy to another. Useful when read-to-update ratio is high.
  3. Propagate the update operation to other copies. The replicas execute the update operation (a.k.a active replication).

Update propagation: Push Vs Pull Protocols

- Pushing updates:
  - Server-based approach: in which updates are propagated regardless whether target asks for it. Multicasting
  - Often used between permanent servers and server initiated replicas (can be used for client caches).
  - Used when a **high degree of consistency** is required and where **read-to-update ratio is relatively high**
- Pulling updates:
  - Client-based approach: in which client polls the server to check whether an update is needed. Unicasting
  - Non-shared client caches.
  - Efficient when **read-to-update ratio is low**.
  - High response time in case of a cache miss

Update Propagation: Leases

- Observation:
  - We can dynamically switch between pulling and pushing using leases.
- Lease is a contract in which the server promises to push updates to the client until the lease expires.
- Lease types (dynamic expiration time)
  - **Age-based:** An object that hasn't changed for a long time, is not likely to change in the near future. So give it a long-lasting lease.
  - **Renewal-frequency based**: The more often a client requests a specific object, the longer the expiration time for that client will be
  - **State-based:** The more loaded the server is, the shorter leases it can issue



Fault Tolerance

One of the important characteristic of a distributed system is that the system will continue to perform (at some level) even if some components / processes / links fail.

Fault Tolerance: Introduction

- It is strongly related to dependable systems.
- Dependability is a term that covers a number of useful requirements for distributed systems including the following

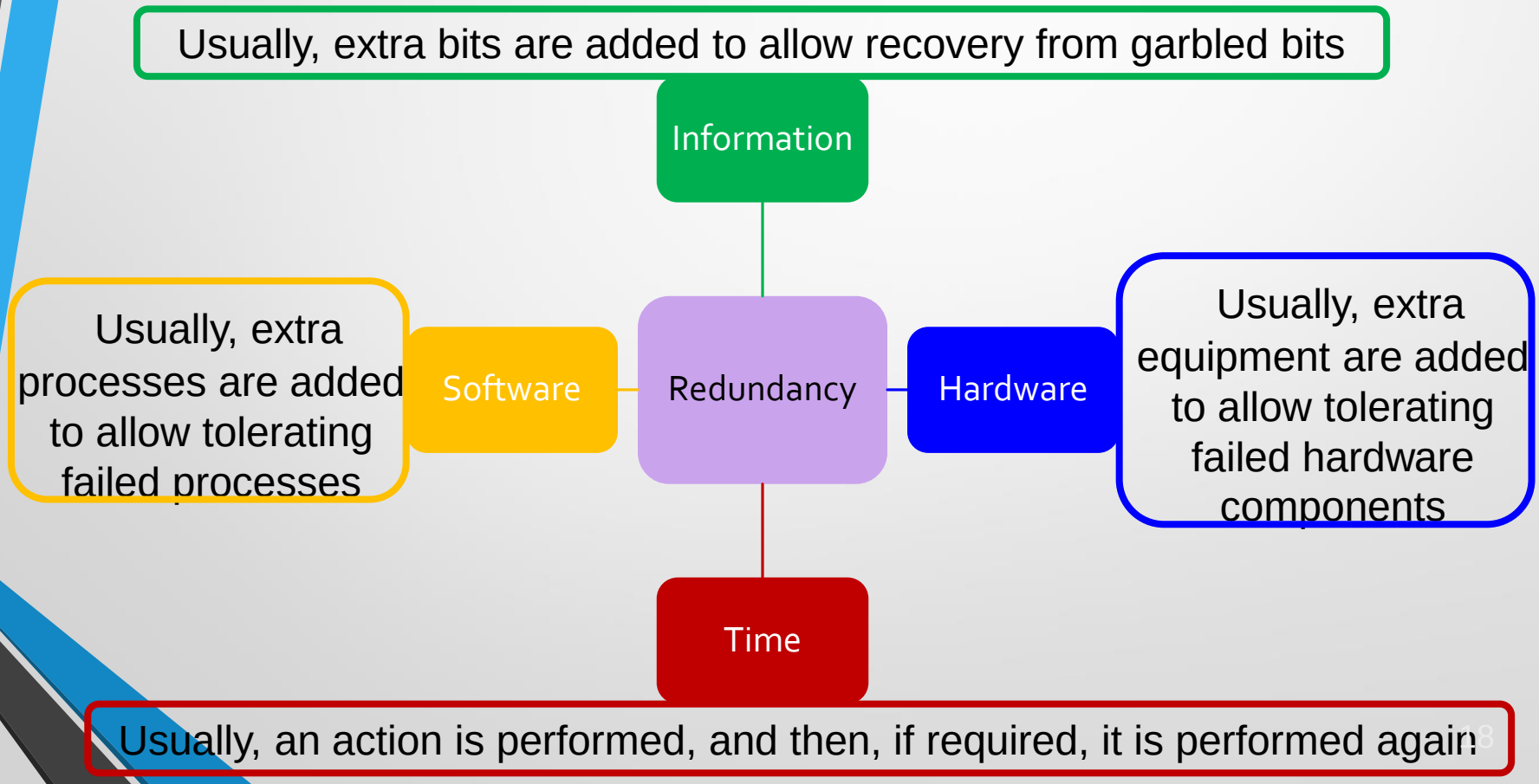
1. **Availability:** a system is ready to be used immediately.
2. **Reliability:** a system can run continuously without failure.
3. **Safety:** a system temporarily fails to operate correctly, nothing catastrophic happens. For example, many process control systems, such as those used for controlling nuclear power plants or sending people into space, are required to provide a high degree of safety
4. **Maintainability**: refers to how easy a failed system can be repaired.

Fault Models: Different types of Failures

Type of failure	Description
Crash failure	A server halts, but is working correctly until it halts
Omission failure	A server fails to respond
Receive omission	A server fails to receive incoming messages
Send omission	A server fails to send messages
Timing failure	A server's response lies outside the specified time interval
Response failure	A server's response is incorrect
Value failure	The value of the response is wrong
State transition failure	The server deviates from the correct flow of control
Arbitrary failure	A server may produce arbitrary responses at arbitrary times

Faults Masking by Redundancy

- The key technique for masking faults is to use *redundancy*



Process Resilience

- How fault tolerance can be achieved in distributed systems
- How can we provide protection against process failures?
  - Replicating processes into groups.
  - Reaching an agreement within a process group when one or more of its members cannot be trusted to give correct answers
- How to detect failures?

Process Resilience (Process groups)

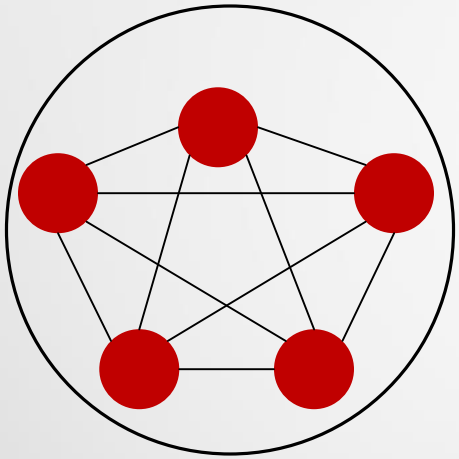
- When a message is sent to a group, all members of the group should receive it (for sender the group appears to be a single and logical process).
- Organize several identical processes into a *group*
- If one process in a group fails,

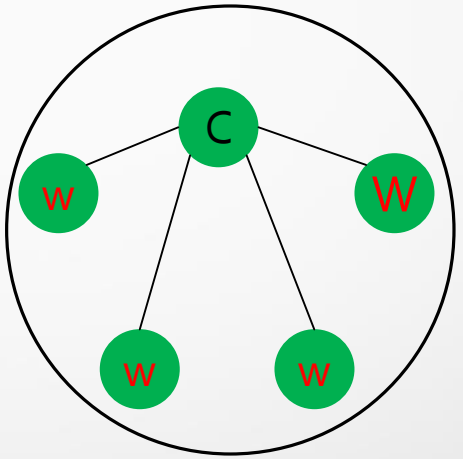
- Hopefully some other process can take over.
- Properties:
  - Process group may be dynamic
  - A process can be a member of several groups at the same time
    - Mechanisms are needed for managing groups and group membership.

Groups are analogous to social organizations.

Group organization

- An important distinction between different groups has to do with their internal structure





Flat Group:
(+) Symmetrical
(+) No single point of failure
(-) Decision making is complicated since it is collective

Hierarchical Group:
(+) Decision making is simple
(-) Asymmetrical
(-) Single point of failure

Group Membership

- When group communication is present, some method is needed for creating and deleting groups, as well as for allowing processes to join and leave groups.
- One possible approach is to have a **group server** to which all these requests can be sent. The group server can then maintain a complete data base of all the groups.
- Another approach is to manage group membership in a distributed way

Membership Management

- **Group Management:** The members of the group are generally dynamic
- The management of the group(creating and deleting groups) and group membership(allowing processes to join or leave groups) is required.
  - IGMP can be used to voluntarily join or exit
- The group management protocols:
  - must rebuilt the group when **many members go down and group cannot work**, either to start election process or using any distributed protocol (token based/nontoken based).
  - **Ensure a synchronous delivery of messages** (when any member joins a group it should receive all messages belongs that group and when it leaves a group, it should not receive any messages that are sent by the group)

Membership Management

Failure Masking and Replication

- By organizing a fault tolerant group of processes, we can protect a single vulnerable process.
- Two approaches to arranging the replication of the group:
  - **Primary (backup) Protocols**
    - A group of processes is organized in a hierarchical fashion in which a primary coordinates all write operations.
    - When the primary crashes, the backups execute some election algorithm to choose a new primary.
  - **Replicated-Write Protocols**
    - These protocols are used in the form of active replication, as well as by means of quorum-based protocols.
    - Solutions correspond to organizing a collection of identical processes into a flat group.

Redundancy requirement

- To understand the amount of redundancy that is required in a process group, a system is said to be *k-fault-tolerant* if it can survive faults in *k* components and still meet its specifications
  - If components fail silently, then having k + 1 of them is enough to provide k-fault-tolerance.
  - If components exhibit Byzantine failures, a minimum of 2k + 1 of them are needed to achieve k-fault-tolerance.
- How can we achieve a *k-fault-tolerant* system?
  - This would require an *agreement protocol* applied to a process group

Agreement in Faulty Systems

- A process group typically requires reaching an *agreement* in:
  - Electing a coordinator
  - Deciding whether or not to commit a transaction
  - Dividing tasks among workers
  - Synchronization
- When the communication and processes:
  - are perfect, reaching an agreement is often straightforward
  - are not perfect, there are problems in reaching an agreement

Agreement in Faulty Systems

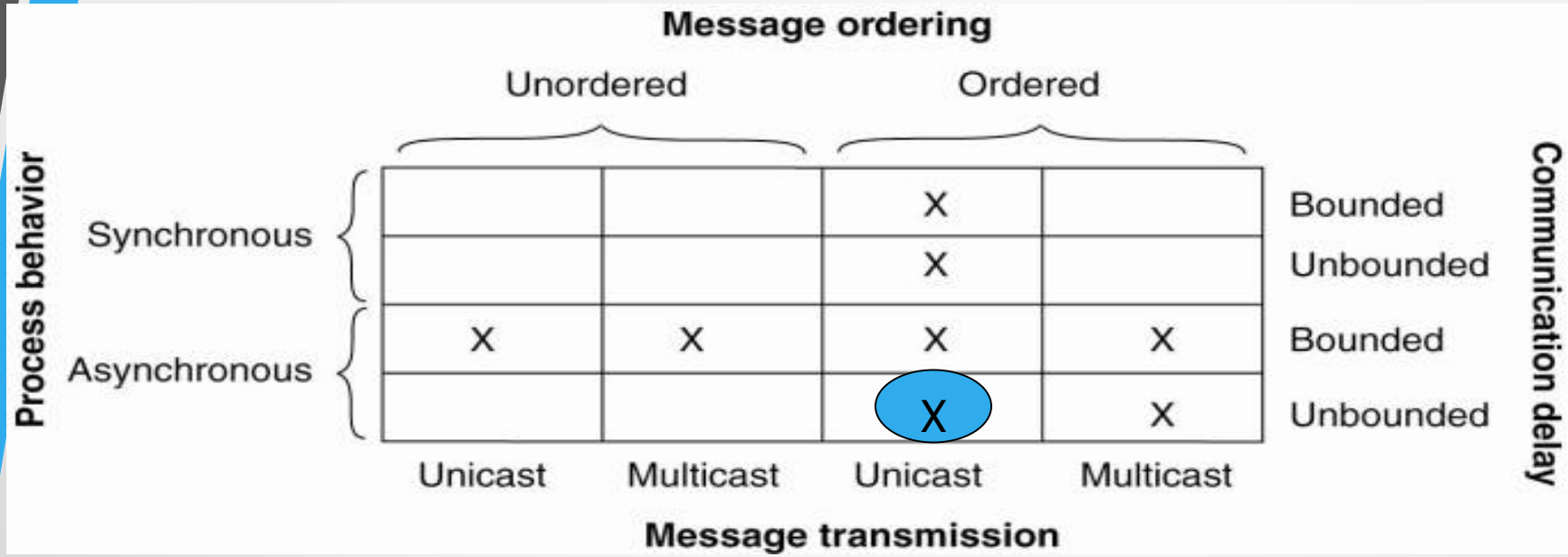
- The goal of distributed agreement is to have all non-faulty processes reach consensus on some issue, and establish that consensus within a finite number of steps

- Different assumptions about the underlying system require different solutions:
  - Synchronous versus asynchronous systems
  - Communication delay is bounded or not
  - Message delivery is ordered or not
  - Message transmission is done through unicasting or multicasting

- Synchronous versus asynchronous systems
  - A system is synchronous if and only if the processes are known to operate in a lock-step mode.
  - Formally, this means that there should be some constant c >= 1, such that if any processor has taken c + 1 steps, every other process has taken at least 1 step.
  - A system that is not synchronous is said to be asynchronous.
- Communication delay is bounded or not.
  - Delay is bounded if and only if we know that every message is delivered with a globally and predetermined max. time.
- Message delivery is ordered or not.
  - In other words, we distinguish the situation where messages from the same sender are delivered in the order that they were sent, from the situation in which we do not have such guarantees.

Message transmission is done through unicasting or multicasting.

Circumstances under which distributed agreement can be reached.



- In all other cases, it can be shown that no solution exists.
- **Note -** most distributed systems in practice assume that processes behave asynchronously, message transmission is unicast, and communication delays are unbounded.
- Usage of ordered (reliable) message delivery is typically required
- The agreement problem has been originally studied by Lamport and referred to as the Byzantine Agreement Problem

Byzantine Agreement Problem

- The goal of Byzantine agreement is that How does a process group deal with a faulty member.
- Lamport assumes:
  - Processes are synchronous
  - Messages are unicast while preserving ordering
  - Communication delay is bounded
  - There are *N* processes, where each process *i* will provide a value v*i* to the others. Let each process construct a vector V of length N, such that
    - If process I is non-faulty, V[i]= vi
    - Else V[i] is undefined
  - There are at most *k* faulty processes

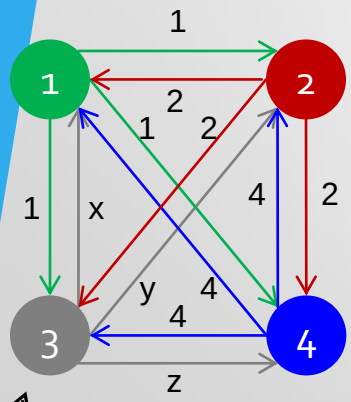
Algorithm (N>=3K)

1. Every non-faulty process i sends vi to every other process using reliable unicasting.
   - Faulty processes may send anything and different values to different processes.
2. The results of the announcements of step 1 are collected together in the form of the vectors.
3. Every process passes its vector from step 2 to every other process.
4. Each process examines the ith element of each of the newly received vectors.
   - If any value has a majority, that value is put into the result vector.
   - If no value has a majority, the corresponding element of the result vector is marked UNKNOWN.

Byzantine Agreement Problem

- **Case I: N = 4 and k = 1**

Step1: Each process sends its value to the others



Faulty process

Step2: Each process collects values received in a vector

1 Got(1, 2, x, 4)
2 Got(1, 2, y, 4)
3 Got(1, 2, 3, 4)
4 Got(1, 2, z, 4)

Step3: Every process passes its vector to every other process

1 Got

(1, 2, y, 4)
(a, b, c, d)
(1, 2, z, 4)

2 Got

(1, 2, x, 4)
(e, f, g, h)
(1, 2, z, 4)

4 Got

(1, 2, x, 4)
(1, 2, y, 4)
(i, j, k, l)

Byzantine Agreement Problem

Step 4:

- Each process examines the *ith* element of each of the newly received vectors
- If any value has a *majority*, that value is put into the result vector
- If no value has a majority, the corresponding element of the result vector is marked UNKNOWN

The algorithm reaches an agreement

Result Vector:
(1, 2, UNKNOWN, 4)

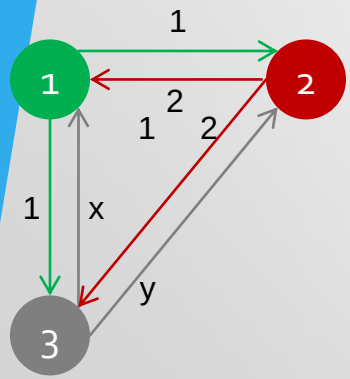
Result Vector:
(1, 2, UNKNOWN, 4)

Result Vector:
(1, 2, UNKNOWN, 4)

Byzantine Agreement Problem

- **Case II: N = 3 and k = 1**

Step1: Each process sends its value to the others



Faulty process

Step2: Each process collects values received in a vector

1 Got(1, 2, x)
2 Got(1, 2, y)
3 Got(1, 2, 3)

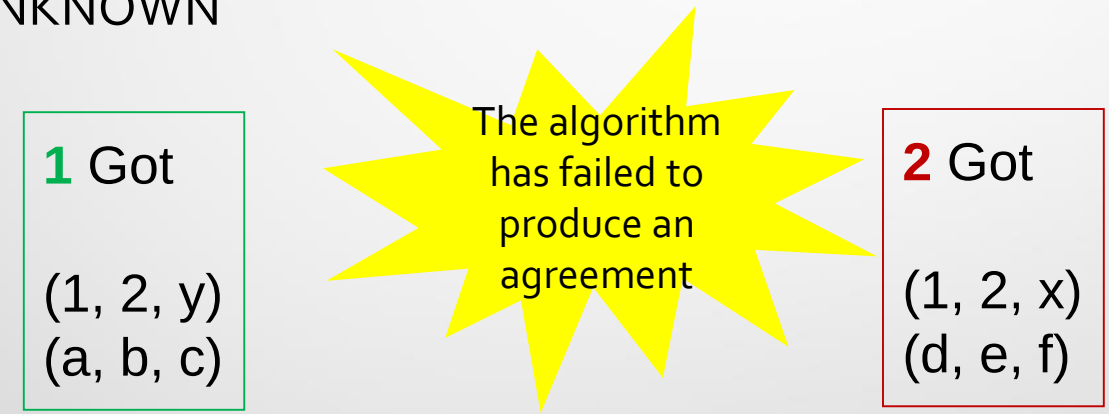
Step3: Every process passes its vector to every other process

1 Got	**2** Got
(1, 2, y) (a, b, c)	(1, 2, x) (d, e, f)

Byzantine Agreement Problem

Step 4:

- Each process examines the *ith* element of each of the newly received vectors
- If any value has a *majority*, that value is put into the result vector
- If no value has a majority, the corresponding element of the result vector is marked UNKNOWN



Result Vector:
(UNKOWN, UNKNOWN, UNKNOWN)

Result Vector:
(UNKOWN, UNKNOWN, UNKNOWN)

Concluding Remarks on the Byzantine Agreement Problem

- In their paper, *Lamport et al.* (1982) proved that in a system with ***k*** faulty processes, an agreement can be achieved only if ***2k+1*** correctly functioning processes are present, for a total of ***3k+1***.
  - i.e., An agreement is possible only if more than two-thirds of the processes are working properly.
- *Fisher et al.* (1985) proved that in a distributed system in which ordering of messages **cannot** be guaranteed to be delivered within a known, finite time, no agreement is possible even if only one process is faulty.

Process Failure Detection

- Before we properly mask failures, we generally need to detect them
- For a group of processes, non-faulty members should be able to decide who is still a member and who is not
- **To detect process failures**, there are two mechanisms:
  - Processes actively send "are you alive?" messages to each other (i.e., *pinging each other*)
  - Processes passively wait until messages come in from different processes

Timeout Mechanism

- In failure detection a ***timeout mechanism*** is usually involved
  - Specify a timer, after a period of time, trigger a timeout
- **Issues:**
  - Timeouts may happen due to various reasons. If any process does not receive reply message within the specified time bond, it is injustice to decide that the process has failed.
  - Due to unreliable networks, simply stating that a process has failed because it does not return an answer to a ping message may be wrong

Failure Considerations

- There are various issues that need to be taken into account when designing a failure detection subsystem:

1. Failure detection can be done by regularly exchanging information with neighbors (e.g., ***gossip-based information dissemination***)

2. A failure detection subsystem should ideally be able to distinguish network failures from node failures

3. When a member failure is detected, how should other non-faulty processes be informed

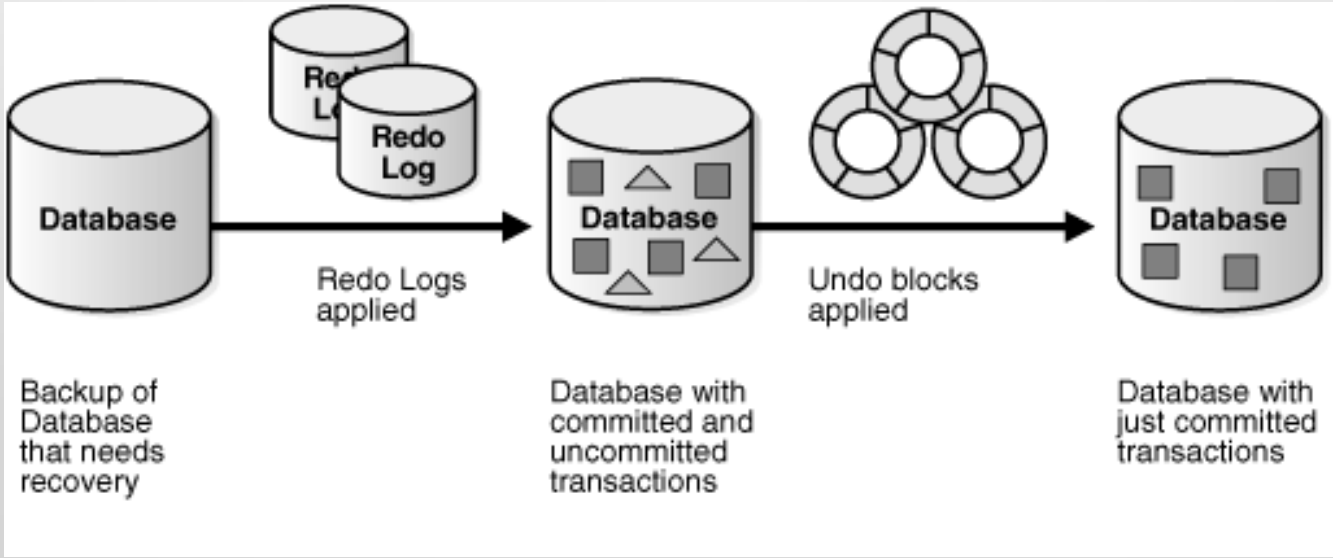
Recovery

- Failure of a system occurs when it does not perform its services in the specified manner.
- Failure recovery is a process that involves restoring an erroneous state to an error free state.
- **Backward recovery:** the process state can be restored to a previous error-free state of the process.
  - To do so it is necessary to record the system's state from time to time, and to restore such a recorded state when things go wrong.
  - Each time (part of) the system's present state is recorded, a checkpoint is to be made.
- **Forward recovery:** If the nature of errors and damages caused by faults can be completely and accurately assessed, then it is possible to remove those errors in the process's state and enable the process to move forward.

- Backward-error recovery is simpler than forwarding error recovery as it is independent of the fault and the errors caused by the fault.
- So, a system can recover from an arbitrary fault by restoring to a previous state.
- The major problems associated with the backward error:
  1. Performance penalty: The overhead to restore a process state to a prior state can be quite high.
  2. There is no guarantee that faults will not occur again when processing begins from a prior state
  3. Some component of the system may be unrecoverable (ex: Bank transactions)
- The forward- error recovery technique incurs less overhead because only those parts of the state that deviate from the intended value need to be corrected.

Recovery with Stable Storage

- Which is designed to survive anything except major calamities such as floods and earthquakes
- Logs are used to store the records. Typically all write records need to be logged. There are two types:
  - write(A) → < Ti, A, old value, new_value>
  - write(A) → <Ti, A, new_value>

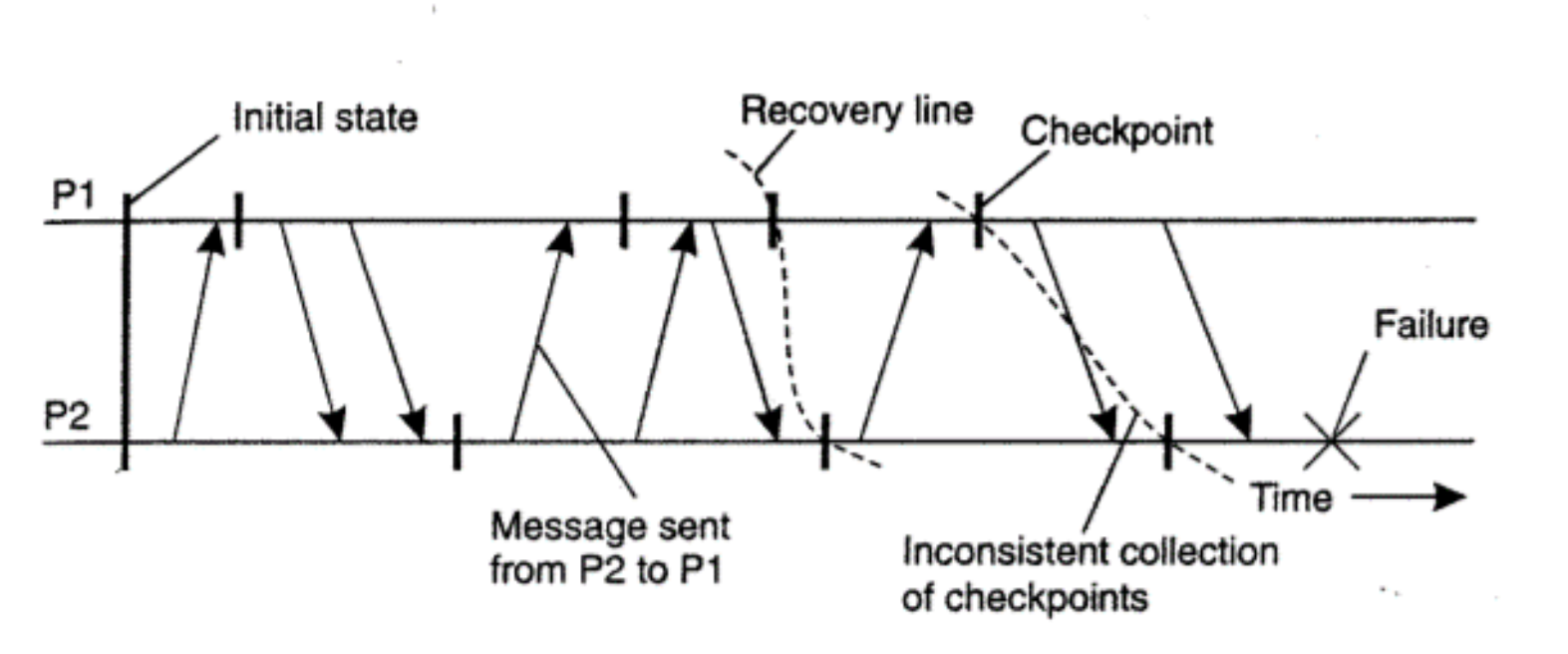


Checkpointing

- In a fault-tolerant distributed system, backward error recovery requires that the system ***regularly saves its state*** onto stable storage.
- Need to record a consistent global state (distributed snapshot).
- In a distributed snapshot, if a process P has recorded the receipt of a message, then there should also be a process Q that has recorded the sending of that message.
- In backward error recovery schemes, each process saves its state from time to time to a locally-available stable storage.
- To recover after a process or system failure is required to construct a consistent global state from these local states.

Checkpointing

- It is best to recover to the most recent distributed snapshot (also called a recovery line).
- In other words, a recovery line corresponds to the most recent consistent collection of checkpoints.

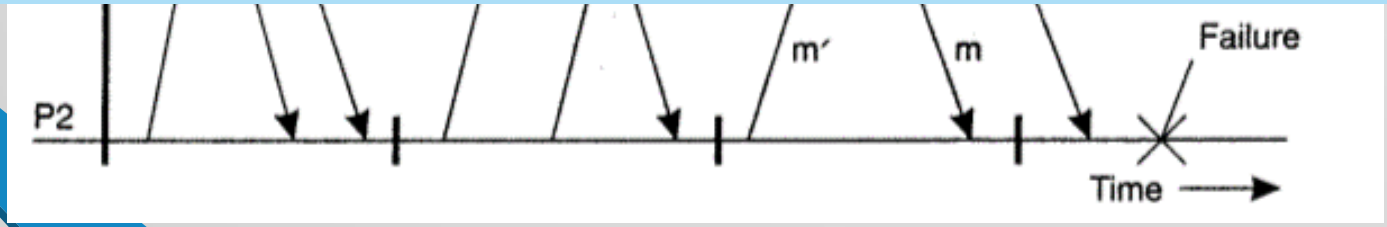


Checkpointing:

Independent Checkpointing & Coordinated Checkpointing

- The distributed nature of checkpointing (in which each process simply records its local state from time to time in an uncoordinated fashion) may make it difficult to find a recovery line.
- To discover a recovery line requires that each process is rolled back to its most recently saved state.
- If these local states jointly do not form a distributed snapshot, further rolling back is necessary. This process of a cascaded rollback may lead to the **domino effect**

P2 needs to be rolled back to an earlier state, then P1 again P2 then P1 ... It turns out that the recovery line is actually the initial state of the system.

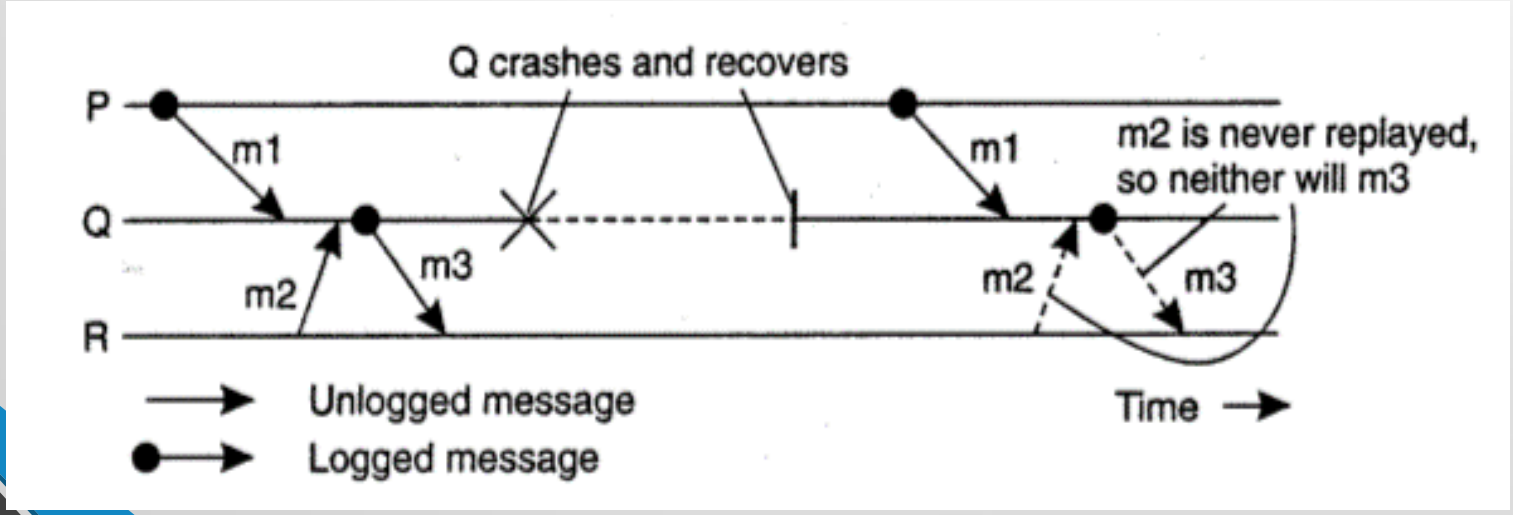


Coordinated Checkpointing

- All processes synchronize to jointly write their state to local stable storage.
- The main advantage is that the saved state is automatically globally consistent so that cascaded rollbacks leading to the domino effect are avoided.
- For implementing this technique we can use a two-phase blocking protocol

Message logging

- Instead of preserving the state of the entire system at once, we can log or record, each change to the state of the system. These changes are easy to identify -- just log each message.
- In the event of a failure, **log** can be **play back** and incrementally restore the system to its original state.



Incorrect replay of messages after recovery, leading to an orphan process

Message logging

- **Synchronous logging.**
- The most intuitive technique for logging a system is synchronous logging.
- Synchronous logging requires that all messages are logged before they are passed to the application.
- In the event of a failure, recovery can be achieved by playing back all of the logs.
- Occasional checkpointing can allow the deletion of prior log entries.

Message logging

- **Synchronous logging.**
- This approach does work, but it is *tremendously* expensive.
- Messages cannot be passed to the application, until the messages are committed to a stable store.
- Stable storage is usually, much slower than RAM.
- Sometimes performance can be improved by carefully managing the I/O device, for example, using one disk for the log file to avoid seek delays, but it is still much slower than a stable storage.

Message logging

- **Asynchronous Logging**
- In order to avoid the performance penalty associated with synchronous logging, we may choose to collect messages in RAM and to write them out occasionally as a batch, perhaps during idle times.
- If we take this approach, we have a similar management problem to that of uncoordinated checkpoints.

Some messages may be logged, while others may not be logged.